React Router

Routing y navegación en SPAs

📝 2026-01 — ⏱️ 60 min.

¿Qué es el Routing en una SPA?

SPA vs Multi-Page App

Multi-Page App (tradicional): cada URL carga una página nueva desde el servidor — el navegador hace una recarga completa.

Single Page App (SPA): el navegador carga el HTML una sola vez. La navegación ocurre en el cliente — JavaScript actualiza el contenido sin recargar.

	Multi-Page	SPA
Navegación	Recarga completa	Sin recarga
Estado	Se reinicia	Se mantiene
URL	Manejada por el servidor	Manejada por JS
Percepción de velocidad	Lenta	Rápida

El problema: si la URL cambia en el cliente, el servidor no sabe qué página mostrar. React Router resuelve esto.

Instalación y Configuración

Instalación de React Router v7

```
npm install react-router
```

React Router v7 unificó `react-router-dom`. Solo se necesita `react-router`.

```
// main.jsx — envolver la app con BrowserRouter
import { BrowserRouter } from 'react-router';
import { createRoot } from 'react-dom/client';
import App from './App';

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

`BrowserRouter` usa la History API del navegador para manejar URLs reales (`/sobre` , `/productos/3`).

Routes y Route

`Routes` elige cuál `Route` renderizar según la URL actual.

```
// App.jsx
import { Routes, Route } from 'react-router';
import Inicio    from './pages/Inicio';
import Sobre     from './pages/Sobre';
import Productos from './pages/Productos';

function App() {
  return (
    <Routes>
      <Route path="/"          element={<Inicio />} />
      <Route path="/sobre"     element={<Sobre />} />
      <Route path="/productos" element={<Productos />} />
    </Routes>
  );
}
```

- `path` — el patrón de URL a coincidir
- `element` — el componente a renderizar
- Solo se renderiza **una** ruta por vez — la primera que coincida

Navegación

Link y NavLink

Nunca usar `<a href>` para navegar dentro de la app — provoca una recarga completa y pierde el estado.

```
import { Link, NavLink } from 'react-router';

// Link — navegación básica
function Navbar() {
  return (
    <nav>
      <Link to="/">Inicio</Link>
      <Link to="/sobre">Sobre nosotros</Link>
      <Link to="/productos">Productos</Link>
    </nav>
  );
}

// NavLink — agrega clase "active" automáticamente al link activo
function Navbar() {
  return (
    <nav>
      <NavLink to="/" className={({ isActive }) => isActive ? 'activo' : ''}>
        Inicio
      </NavLink>
      <NavLink to="/productos">Productos</NavLink>
    </nav>
```

Rutas Dinámicas

Parámetros en la URL con :param

Las rutas dinámicas permiten URLs con datos variables como `/productos/42` o `/usuarios/maria`.

```
// Definir la ruta dinámica
<Route path="/productos/:id" element={<DetalleProducto />} />

// En el componente — leer el parámetro con useParams
import { useParams } from 'react-router';

function DetalleProducto() {
  const { id } = useParams(); // "42" si la URL es /productos/42
  return <h1>Producto #{id}</h1>;
}

// Navegar a una ruta dinámica
<Link to="/productos/42">Ver producto</Link>

// o con variable:
<Link to={`/productos/${producto.id}`}>Ver producto</Link>
```

Hooks de Navegación

useNavigate y useLocation

```
import { useNavigate, useLocation } from 'react-router';

// useNavigate — navegar por código (después de un submit, por ejemplo)
function FormularioLogin() {
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    await login(datos);
    navigate('/dashboard');               // ir a una ruta
    // navigate(-1);                      // volver atrás (botón Back)
    // navigate('/login', { replace: true }); // reemplaza el historial
  }
}

// useLocation — leer la URL actual
function ComponenteAnaliticas() {
  const location = useLocation();
  // location.pathname → "/productos/42"
  // location.search   → "?color=rojo&talla=L"
  // location.hash     → "#descripcion"

  useEffect(() => {
    registrarPageView(location.pathname);
```

Rutas Anidadas y Layouts

Nested Routes y Outlet

Las rutas anidadas permiten que un componente padre muestre las rutas hijas en su interior.

```
// Rutas anidadas en App.jsx
<Routes>
  <Route path="/dashboard" element={<LayoutDashboard />}>
    <Route index         element={<ResumenDashboard />} />  {/* /dashboard */}
    <Route path="perfil" element={<Perfil />} />            {/* /dashboard/perfil */}
    <Route path="config" element={<Configuracion />} />     {/* /dashboard/config */}
  </Route>
</Routes>

// LayoutDashboard.jsx — usa <Outlet /> para renderizar la ruta hija
import { Outlet, NavLink } from 'react-router';

function LayoutDashboard() {
  return (
    <div className="layout">
      <aside>
        <NavLink to="/dashboard">Resumen</NavLink>
        <NavLink to="/dashboard/perfil">Perfil</NavLink>
        <NavLink to="/dashboard/config">Configuración</NavLink>
      </aside>
      <main>
        <Outlet /> {/* Aquí se renderiza la ruta hija activa */}
```

Ruta 404

Catch-All y Página No Encontrada

```
// path="*" captura cualquier URL que no haya coincidido antes
<Routes>
  <Route path="/"          element={<Inicio />} />
  <Route path="/productos" element={<Productos />} />
  <Route path="/productos/:id" element={<DetalleProducto />} />
  <Route path="*"          element={<PaginaNoEncontrada />} />
</Routes>

// PaginaNoEncontrada.jsx
import { Link } from 'react-router';

function PaginaNoEncontrada() {
  return (
    <div>
      <h1>404 — Página no encontrada</h1>
      <p>La dirección que buscas no existe.</p>
      <Link to="/">Volver al inicio</Link>
    </div>
  );
}
```

Importante para deploy: los servidores estáticos necesitan configuración para redirigir todas las URLs al `index.html`. Vercel lo hace automáticamente para proyectos Vite.

Diseño de URLs

URLs Legibles — Sin IDs, Con Slugs

La URL es parte de la interfaz. Un usuario debe poder leerla y entender dónde está.

```
❌ URLs que nadie puede leer ni compartir:
/productos/a1b2c3d4-e5f6-7890-abcd-ef1234567890
/users/83729/posts/4921?ref=nav&src=email&utm_id=xyz

✅ URLs limpias y predecibles:
/productos/tenis-nike-air-max
/blog/como-aprender-css-en-2025
/dashboard/configuracion
/u/maria-garcia
```

Slug: versión legible de un título, en minúsculas, palabras separadas con guiones.

```
// Generar un slug desde un título
function crearSlug(titulo) {
  return titulo
    .toLowerCase()
    .normalize('NFD')                  // separar tildes
    .replace(/[\u0300-\u036f]/g, '')   // quitar tildes
    .replace(/[^a-z0-9\s-]/g, '')      // quitar caracteres especiales
    .trim()
    .replace(/\s+/g, '-');             // espacios → guiones
```

Resumen — React Router

- ✅ Instala `react-router` y envuelve la app con `<BrowserRouter>`
- ✅ Usa `<Routes>` + `<Route path="..." element={...}>` para definir la navegación
- ✅ Usa `<Link to="...">` y `<NavLink>` para navegar — nunca `<a href>` para rutas internas
- ✅ Rutas dinámicas con `:param` → léelas con `useParams()`
- ✅ Navega por código con `useNavigate()` tras acciones (submit, login, etc.)
- ✅ Usa rutas anidadas + `<Outlet>` para compartir layouts entre páginas
- ✅ Agrega `<Route path="*">` al final para capturar URLs inválidas (404)
- ❌ No uses `<a href>` para rutas internas — recarga la app y pierde el estado